

MGPS Local Wireless Gateway Architectures

1. Purpose and Scope

At a typical hospital bedside, pipes for several gases converge in one location. Each gas line is monitored by its own set of roughly six sensors, so a single bedside location could have on the order of 30 sensors in total. Rather than giving every sensor its own wireless radio, the architectures in this document share one idea: group each gas's sensor set onto its own board with wired (I2C/CAN/etc.) communication to a local microcontroller, then bridge that wired data to the IoT hub through a local wireless gateway. This keeps most devices off the air entirely, which directly reduces RF traffic, congestion, and the number of radios that need to be managed. This document covers four different architectures, compares them directly, and proposes additional variations worth considering.

2. Architecture A - Shared Bus to a Single Wireless Transceiver (CAN Bus)

Each gas board (Oxygen, CO2, N2, Vacuum, Air) has its own “Decent MCU” that reads its six local sensors over a wired interface. All five board MCUs share a single CAN bus running to the central hub, which contains only a wireless transceiver — no local MCU of its own. The transceiver simply forwards whatever arrives on the CAN bus out over the air.

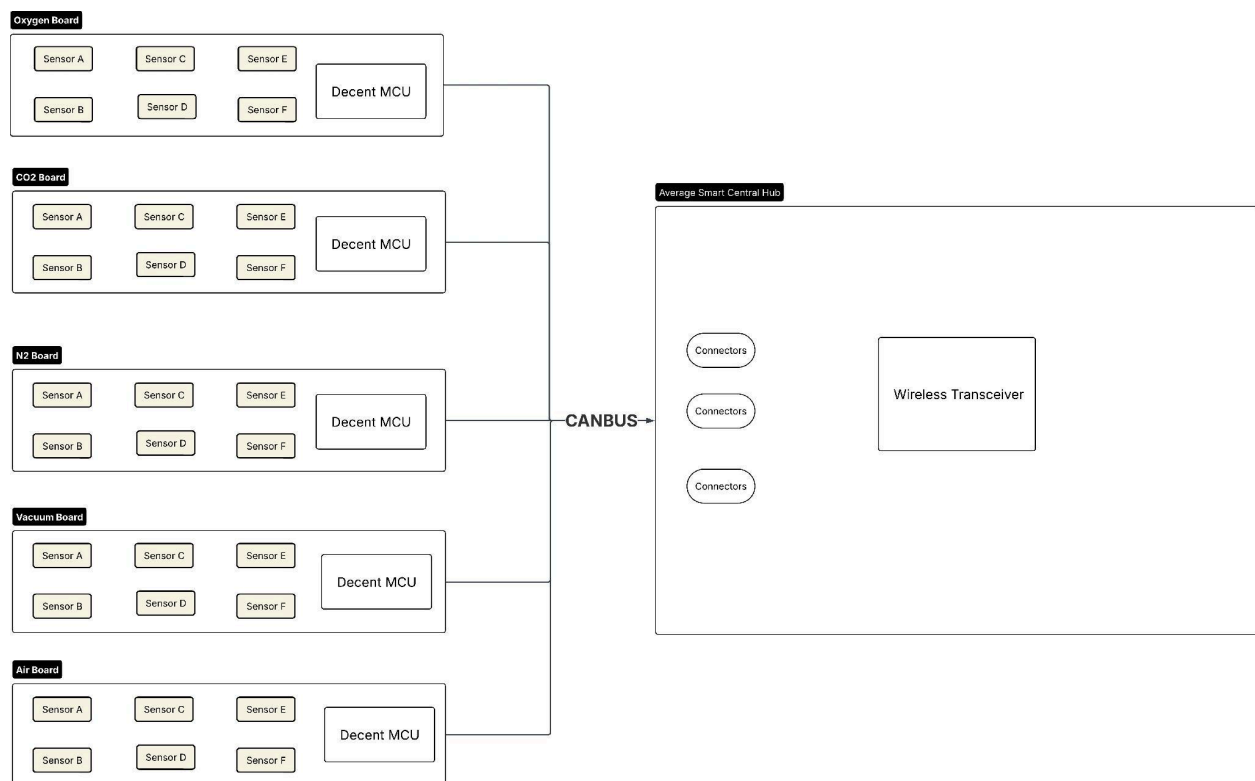


Figure 1. Five gas-board MCUs share one CAN bus into a hub with a bare wireless transceiver (no hub-side MCU).

Pros

- CAN is a mature, robust, differential-signaling bus designed for noisy industrial/automotive environments, a good match for a mechanical room full of metal pipework and equipment.
- CAN's multi-drop topology means adding or removing a gas board doesn't require rewiring point-to-point links, since every board taps the same bus.
- Built-in CAN error detection/arbitration gives some protection against a single bad node corrupting the whole bus.

- Hub hardware is minimal. A transceiver alone is cheaper and simpler to certify than a transceiver-plus-MCU hub.

Cons

- Pushing protocol framing, security, and queuing entirely onto a “bare” transceiver is unusual since most wireless transceivers expect a host MCU to hand them already-formatted packets. In practice this hub likely still needs at least a thin MCU or glue logic, even if it's not drawn here.
- The CAN bus itself is a shared resource, so a wiring fault, short, or stuck node can take down communication for every gas board at that bedside simultaneously.
- Five separate “Decent MCUs” (one per gas) is more total silicon than necessary if a single MCU could read all 30 sensors directly (see Architecture D).

3. Architecture B - Point-to-Point Wiring to a Two-Chip Hub (Decent MCU + Transceiver)

Each gas board has its own “Budget MCU”, a step down from Architecture A's “Decent MCU,” since these boards now only need to digitize sensors and forward data, not manage a shared bus. Each board connects individually (point-to-point, not multi-drop) into the central hub. The hub itself contains a “Decent MCU” that aggregates the incoming data and hands it to a separate wireless transceiver chip for transmission.

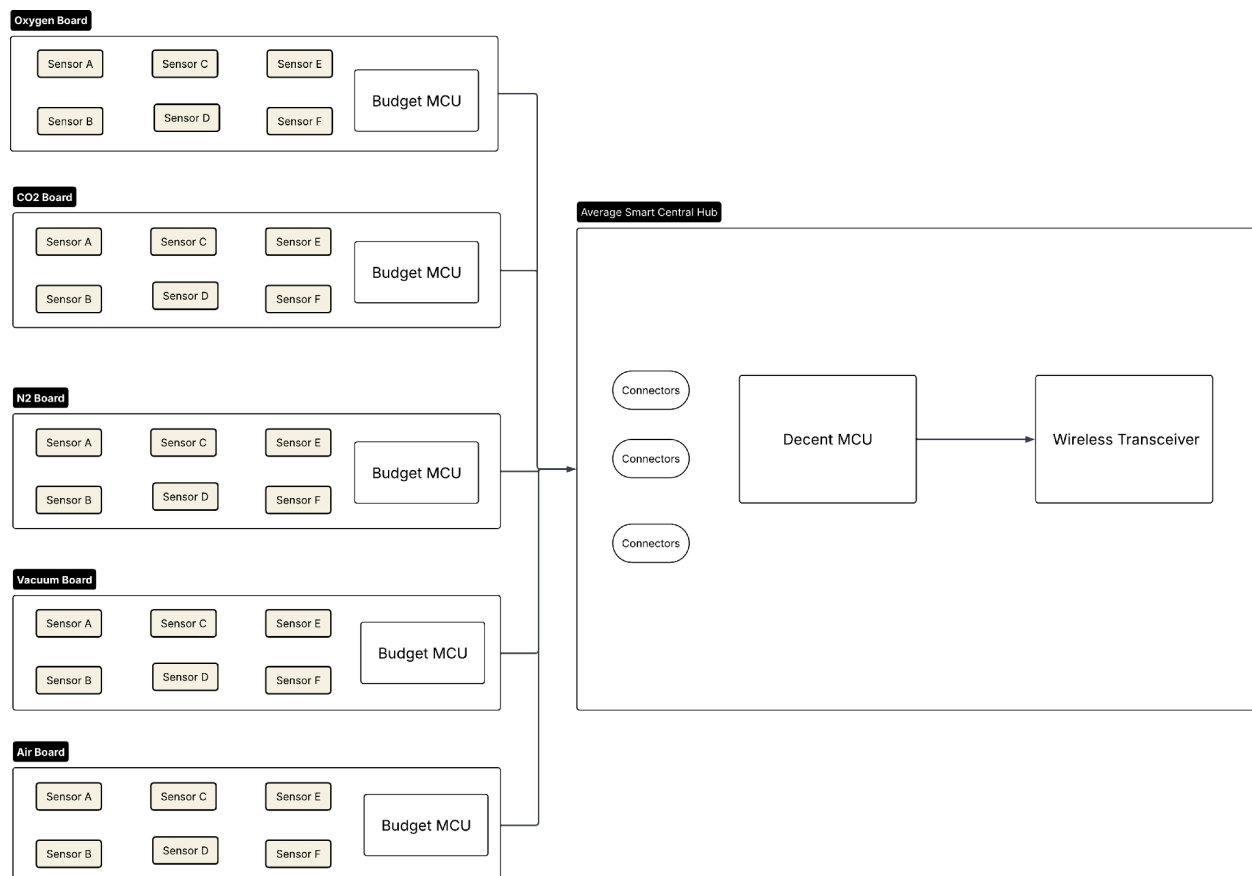


Figure 2. Five Budget-MCU boards wire point-to-point into a hub containing a Decent MCU plus a separate wireless transceiver.

Pros

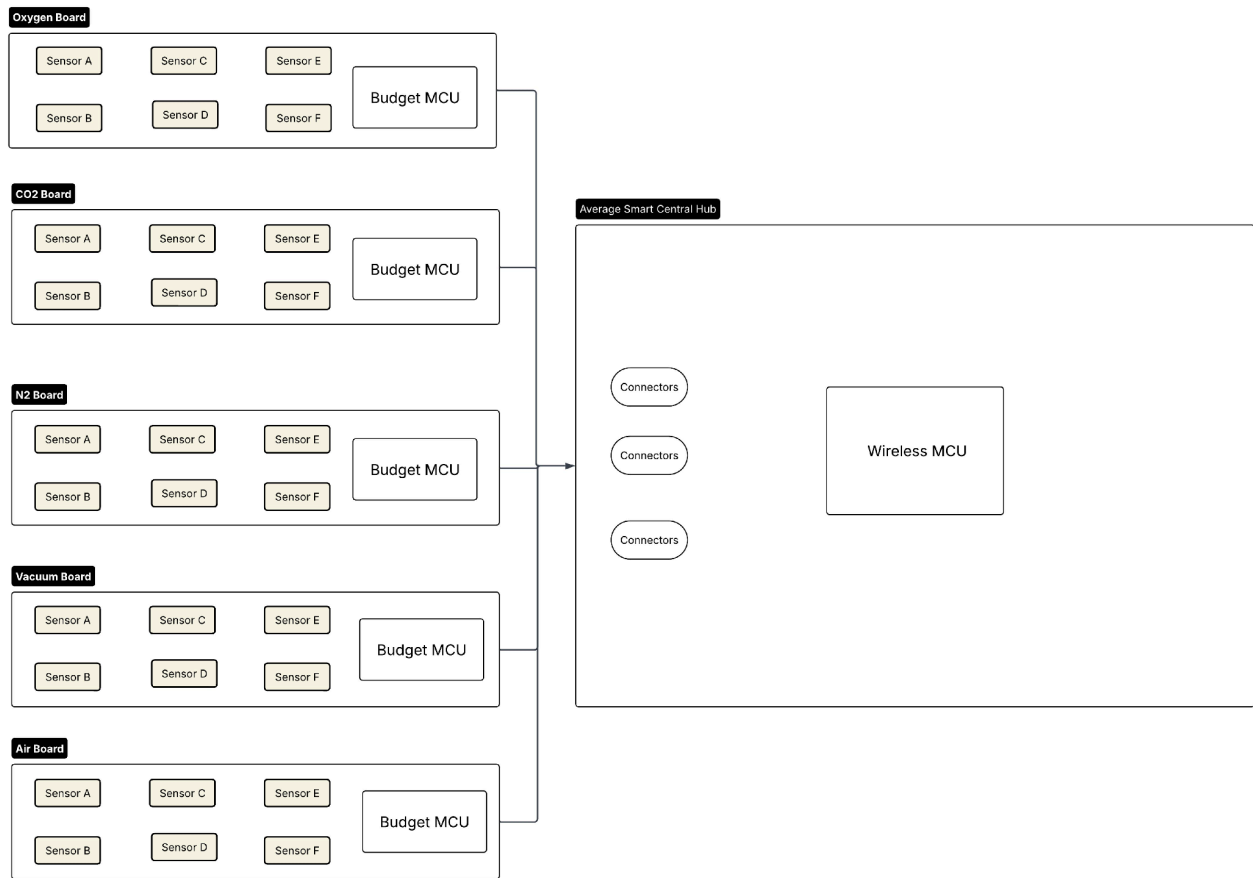
- Splitting “Decent MCU” (protocol/aggregation logic) from “Wireless Transceiver” (RF only) at the hub is a conventional, well-supported pattern. Many wireless stacks (Wi-SUN, mioty, LoRaWAN via SX1262) are built this way, so this is the most straightforward to implement with off-the-shelf parts.
- Point-to-point wiring avoids CAN's shared-bus single point of failure, since one board's wiring fault doesn't directly affect the others, only its own connection to the hub.
- Budget MCUs on each gas board reduce per-board cost relative to Architecture A's five “Decent MCUs.”

Cons

- Point-to-point wiring means five separate physical runs from the boards to the hub, rather than one shared bus, which is more individual cables/connectors to route and maintain at a bedside.
- Two chips at the hub (MCU + transceiver) means more hub-side cost and board area than a single combined wireless MCU, and one more interface (SPI/UART between them) that has to be validated.
- “Budget MCU” capability is not yet defined, if these need to do any local sensor-specific processing (e.g., calibration, the Sensor Controller pattern already used on the CC1354P10), this may be a significant constraint.

4. Architecture B Variant - Same Wiring, Combined Wireless MCU at the Hub

This is a direct variant of Architecture B: the gas boards and their point-to-point wiring into the hub are identical, but the hub itself uses a single combined “Wireless MCU” (a chip with both a capable core and an integrated radio, like the CC1354P10 itself) instead of a separate MCU-plus-transceiver pair.



High end, High Cost

Figure 3. Same point-to-point wiring as Architecture B, but the hub combines compute and radio into a single Wireless MCU.

Pros

- Fewer hub-side parts and one less inter-chip interface to design and debug, compared to the two-chip hub in Architecture B.
- If the CC1354P10 (or similar SimpleLink-class part) is used at the hub, this reuses exactly the chip and SDK already chosen for the project, which keeps tools and firmware consistent across gas boards and hub.
- Likely a smaller hub footprint, which matters if the hub needs to fit into existing bedside infrastructure rather than being a standalone enclosure.

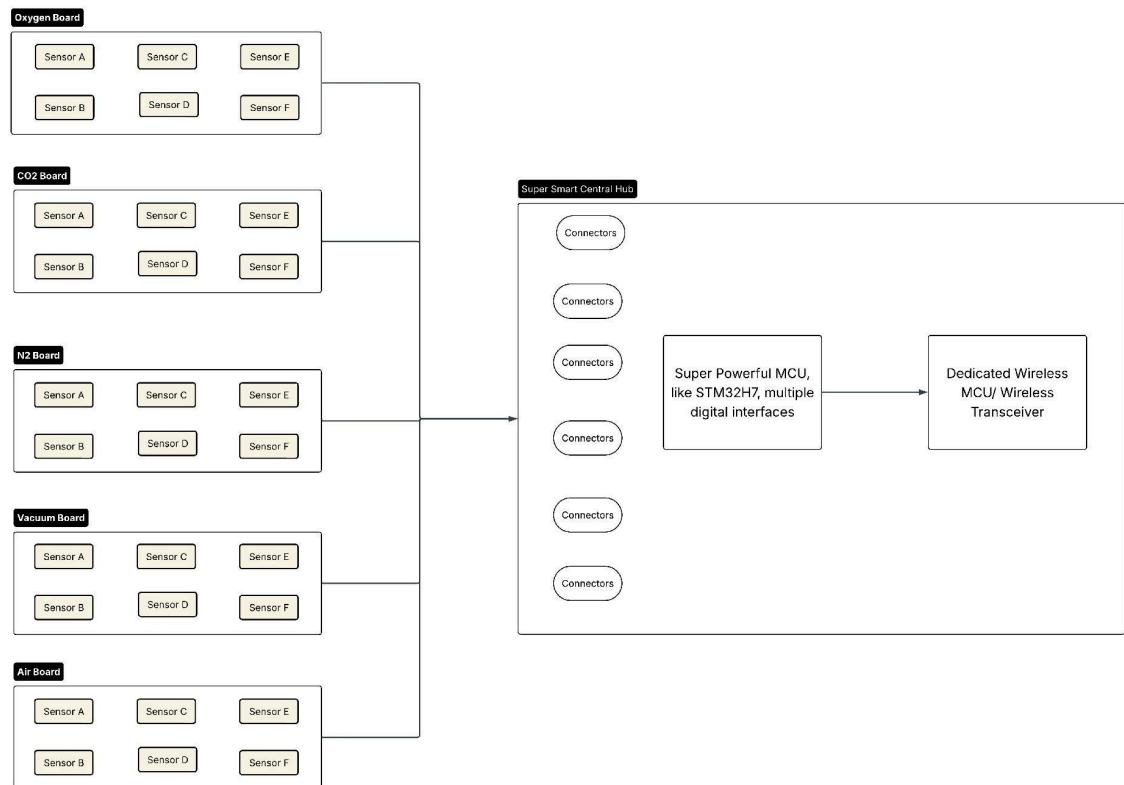
Cons

- A single combined chip ties the hub's compute capability to whatever a wireless-capable part offers; if more aggregation/processing power is needed at the hub than a wireless MCU provides, there's no easy way to add it without a second chip anyway.
- Less flexibility to upgrade the radio independently of the compute, or vice versa, compared to the two-chip approach.
- Still inherits the point-to-point wiring tradeoff from Architecture B, so there are still around five individual runs per bedside.

5. Architecture C - High-End Hub with a Powerful MCU and Dedicated Wireless Module

This is the lowest possible cost option overall, despite using the most capable hub. The gas boards in this version have no local MCU at all, sensors are wired directly into the hub's connectors. The hub contains a “Super Powerful MCU” (the example given was an STM32H7-class part with multiple digital interfaces) that reads all the sensors directly, feeding into a “Dedicated Wireless MCU / Wireless Transceiver” for transmission.

Could be similar to our Test Board, just with more sensors



Lowest Possible Cost

Figure 4. All sensors wire directly into a single high-end hub; no per-gas-board MCU is needed.

Pros

- Removing per-board MCUs entirely is the most direct way to cut total silicon cost and firmware surface area since there is exactly one compute chip to program and maintain (plus the radio), instead of five-plus-one.
- A powerful MCU like the STM32H7 has enough digital interfaces (multiple I2C/SPI/UART) to directly address ~30 sensors without needing the Sensor Controller-style offloading currently used on the CC1354P10.
- Centralizing all sensor reads in one place simplifies timing coordination across gas boards (relevant for Architecture D's air-timing concern) since one MCU sees all sensors at once rather than five MCUs needing to coordinate with each other.

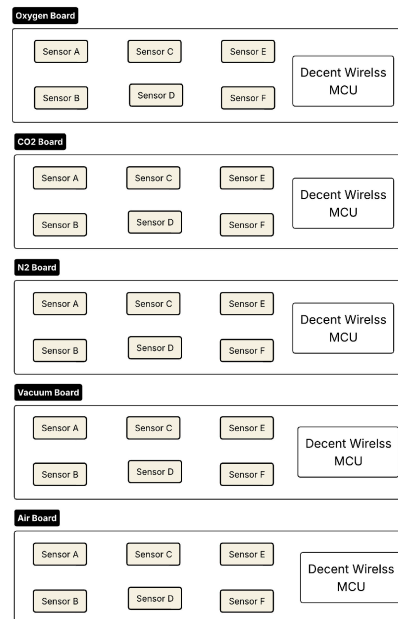
Cons

- This is the architecture with the most classic single point of failure: if the hub MCU or its wiring fails, every gas board at that bedside loses monitoring simultaneously; there is no local fallback the way Architectures A/B/D each retain at least a board-level MCU.
- “Direct wiring” from up to 30 sensors into one hub implies a large number of physical connectors/cable runs converging on a single enclosure, which may be harder to route cleanly at a real bedside.
- A part like the STM32H7 is a more powerful, more expensive per-unit MCU than the CC1354P10.
- Tradeoff: Lowest possible cost, but also single-point-of-failure risk.

6. Architecture D - Fully Distributed: A Wireless MCU per Gas Board

The most decentralized option. Each gas board keeps its own “Decent Wireless MCU” (a combined compute-plus-radio chip, same concept as the Architecture B variant) and transmits directly. There is no shared local hub or wired bus between boards at all. The two main points are that the boards will need to coordinate Air timing with each other, and there is truly no single point of failure.

Need to Coordinate Air timing



Truly No Single Point of Failure

Figure 5. Each gas board has its own wireless MCU and transmits independently, no shared hub or wired bus.

Pros

- No shared point of failure between gas boards: a fault on the Oxygen board's MCU, wiring, or radio has zero effect on the CO2, N2, Vacuum, or Air boards.
- No wired bus or point-to-point hub wiring to route at all between boards — each board is a self-contained wireless unit, which may simplify physical installation at a bedside with awkward geometry.
- Matches the architecture the team has already been developing toward (a wireless-capable MCU per sensor cluster).

Cons

- This is the architecture the local-gateway approach was specifically meant to avoid: five separate radios transmitting per bedside is exactly the wireless device count and RF traffic the new architecture designs were trying to reduce.

- The boards are not fully independent in practice: must coordinate air timing still.
- Five times the wireless transceivers per bedside means five times the radio hardware cost, five times the certification/compliance surface, and the highest RF congestion footprint of any option here.
- If the building has many bedsides, this architecture scales the worst: total wireless node count grows linearly with (bedsides × gases), where the other three architectures can keep wireless node count closer to just (bedsides).

7. Direct Comparison

Dimension	A: CAN + Transceiver	B: P2P + 2-Chip Hub	B-var: P2P + Wireless MCU	C: Direct-Wire Super Hub	D: Per-Board Wireless
Wireless transceivers per bedside	1	1	1	1	5 (one per gas)
Per-gas-board MCU?	Yes (Decent)	Yes (Budget)	Yes (Budget)	No — direct wire to hub	Yes (Decent Wireless)
Hub compute	None shown (transceiver only)	Decent MCU + transceiver	Single Wireless MCU	Super Powerful MCU + wireless module	N/A — no hub
Board-to-hub wiring	Shared CAN bus	Point-to-point	Point-to-point	Direct sensor wiring	None — wireless only
Single point of failure	Shared CAN bus	Hub (2-chip)	Hub (1-chip)	Hub (highest — all sensors converge)	None
Relative hub complexity	Lowest (if transceiver-only is viable)	Moderate	Lower than B	Highest	N/A
Relative total BOM	Moderate	Moderate	Moderate-low	Lowest (fewest total MCUs)	Highest (5 radios)

8. Additional Architectures Worth Considering

8.1 Mesh-of-Hubs (Bedside Hub-to-Hub Relay)

On a hospital floor, some bedsides may be too far from the nearest gateway or blocked by equipment. A natural extension of these architectures is to let bedside hubs relay for each other in a mesh (using Wi-SUN or mioty's native mesh/star capability on the CC1354P10) rather than requiring every hub to reach the gateway in one hop. This preserves the “one radio per bedside” benefit of Architectures A/B/B-variant while adding the obstruction-resilience benefit that Architecture D gets only by brute-force radio count.

8.2 Wired Cluster Head, Sub-Metered by Gas

A hybrid between Architecture C and the per-board approach: keep one small, cheap MCU per gas board (as in B/B-variant) purely for local sensor digitization and basic validity checks, but have all five boards report over a short wired bus to a single bedside hub that does both aggregation and the radio work (merging B-variant's hub design with A's shared-bus wiring). This keeps the single point of failure at the hub level (like B/C) while avoiding Architecture A's open question of whether a bare transceiver can really replace a full hub MCU.

8.3 Dual-Radio Hub for Redundancy

Borrowing the “fully loaded” idea from the wireless protocol comparison document: a bedside hub (Architecture B-variant or C) could carry two radios, for example Wi-SUN/mioty on the CC1354P10's native radio plus an external LoRa/SX1262 path, so that if one wireless link degrades due to local RF conditions, the hub can fail over to the other. This adds hub cost and firmware complexity but directly addresses Architecture C's biggest weakness (a hub failure takes out the whole bedside) without going all the way to Architecture D's five-radios-per-bedside cost.

8.4 Tiered Criticality Split

Not every gas line may carry the same safety weight at every bedside. A tiered design could put the Oxygen board (often the most safety-critical) on its own independent wireless MCU (Architecture D's pattern, but only for one gas) while CO₂, N₂, Vacuum, and Air share a single lower-cost hub (Architecture B/B-variant). This concentrates the cost and redundancy of full independence only where it matters most, rather than applying it uniformly across all five gases.